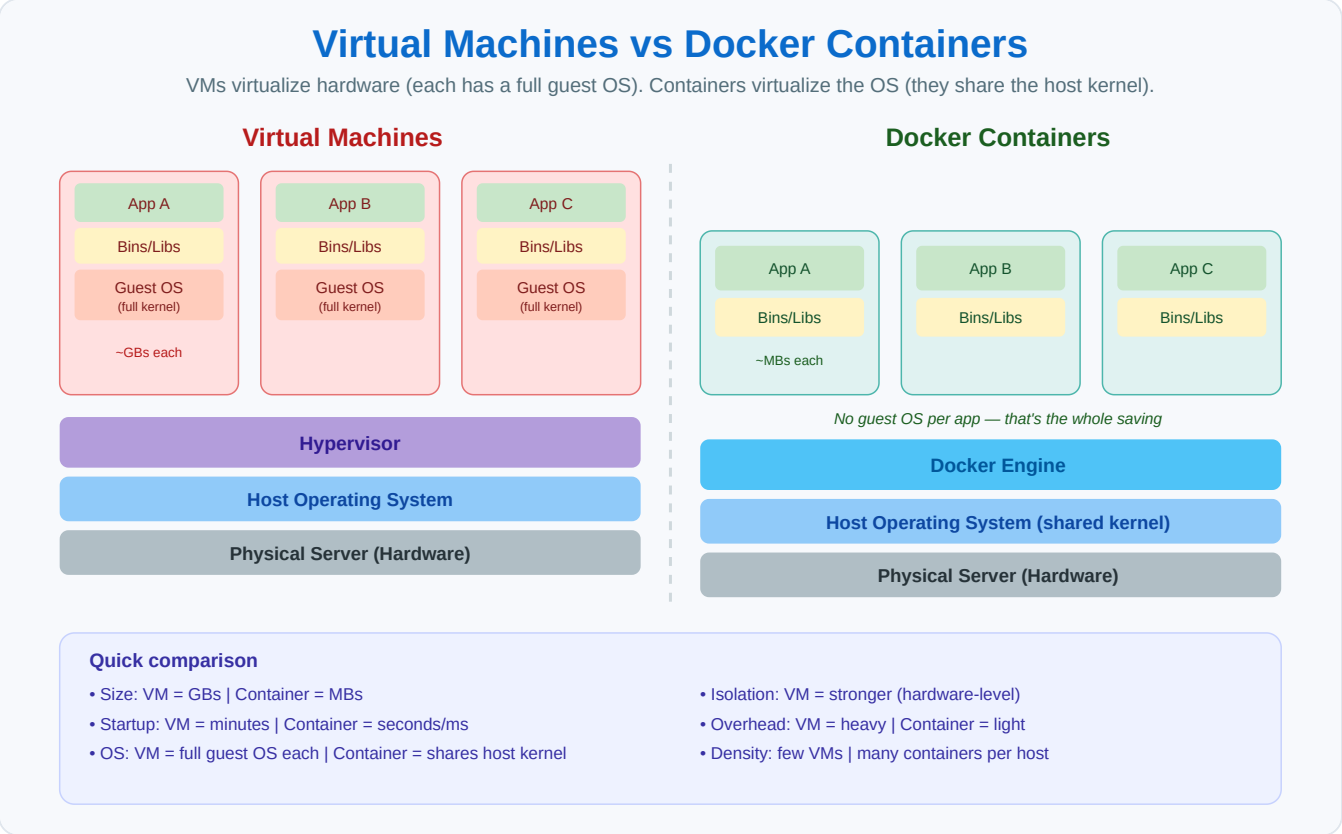
🐳 Docker vs Virtual Machine

> Topic-only study notes • Taught in 3 stages + Interview Corner • With diagram Examples tailored to a JavaScript / Next.js / MongoDB stack.

Learner: Akshay • **Date:** 6 August 2026

Docker vs Virtual Machine



🟢 Beginner

Both VMs and containers run multiple isolated apps on one machine, differently.

- **VM = separate houses** 🏠 — each has its own foundation, plumbing, walls (a *full* OS). Isolated but heavy/slow.
- **Container = apartments in one building** 🏢 — share the building's structure (host OS) but each is private. Light, many fit.

In the diagram, the VM side carries a **full Guest OS per app** (heavy: GBs, boots in minutes). The Docker side has **no guest OS per app** — all share the host OS via the Docker Engine (light: MBs, starts in seconds).

One line: A VM virtualizes an entire *computer*; a container virtualizes just the *app's environment*, sharing the host OS.

🟡 Working Developer

Aspect	Virtual Machine	Docker Container
Size	Gigabytes (full OS)	Megabytes (app + libs)
Startup	Minutes	Seconds / ms
OS	Full guest OS each	Shares host kernel
Resources	Heavy	Light
Per machine	A handful	Dozens–hundreds
Isolation	Very strong (hardware)	Strong (process)
Portability	Large, slow to move	Small, easy to ship

Running Next.js + Express + MongoDB + Redis as VMs = 4 full OSes (~8GB+ RAM). As containers = 4 lightweight processes sharing one OS, starting in seconds — why dev, CI, and microservices use containers.

Not enemies: cloud servers are usually VMs (e.g. EC2), and you run **containers inside** those VMs — VM isolation + container speed. (Docker Desktop on Windows/Mac also runs a small Linux VM under the hood.)

🔴 Expert

- **What's virtualized:** VM virtualizes **hardware** via a **hypervisor**; each guest has its own kernel. Container virtualizes the **OS** — isolated processes on the host kernel via **namespaces** + **cgroups** (no guest kernel).
- **Kernel sharing → OS constraint:** a Linux container needs a Linux kernel; can't natively run Windows containers on Linux (why Docker Desktop uses a Linux VM).
- **Isolation boundary:** container escape is a more realistic threat (shared kernel = larger attack surface); VMs have a smaller, hardened hypervisor boundary. Hybrids: **gVisor**, **Kata Containers**, **Firecracker** (container speed + VM-like isolation).
- **Density/efficiency:** no duplicated guest OS → hundreds of containers where a few VMs fit — the economics behind cloud-native + Kubernetes.
- **When to use which:** Containers → microservices, stateless apps, CI/CD, dev, fast scaling. VMs → different OS than host, strong isolation for untrusted/multi-tenant, legacy/full-OS apps, kernel customization. Both → containers inside VMs (standard cloud pattern).

🎯 Interview Corner

- **Main difference?** VM virtualizes hardware + full guest OS (hypervisor); container virtualizes OS, shares host kernel, isolates the process. VM heavier/more isolated; container lighter/faster.
- **Why lighter?** No full guest OS — shares host kernel → MBs not GBs, seconds not minutes.
- **Less secure?** Generally yes at the boundary (shared kernel, larger attack surface); VMs isolate via hypervisor. Mitigations: gVisor, Kata, Firecracker.

- **Windows container on Linux?** Not natively — shared kernel; Docker Desktop uses a Linux VM.
- **Do containers replace VMs?** No — complementary; containers commonly run *inside* VMs.
- ⚠️ **Traps:** calling a container "a lightweight VM" (different mechanism); claiming containers are strictly better (VMs win on isolation/cross-OS); forgetting the shared-kernel fact that explains size, speed, OS limit, and security.